DevOps Shack

Engineering Excellence Series

Git Branching Strategy

Development → QA → Production

Feature • Bugfix • Hotfix • QA • Main

main	qa	feature	bugfix	hotfix

1. Branching Strategy Overview

A well-defined Git branching strategy is the backbone of a healthy DevOps pipeline. It determines how code moves from a developer's laptop all the way to production — ensuring every change is tested, reviewed, and traceable. This document describes the branching model used by DevOps Shack, built around five branch types that map cleanly onto two environments: Development/Testing (QA cluster) and Production.

Core Philosophy

main is always production-ready. **qa** is always stable enough for testing. Feature work never touches either directly — it flows through short-lived branches with mandatory PR reviews and automated gates.

2. Branch Types — Quick Reference

Branch	Created from	Merged into	Environment	Purpose
main	— (permanent)	—	**PROD**	Live production code. Always deployable.
qa	main	main	**QA / Staging**	Integration & regression testing hub.
feature/*	qa	qa	**DEV (local)**	New functionality developed by engineers.
bugfix/*	qa	qa	**QA**	Fixing defects found during QA testing.
hotfix/*	main	main + qa	**PROD (emergency)**	Critical production fix, bypasses qa flow.

© 2025 DevOps Shack. All rights reserved.

3. Visual Branch Flow Diagram

The diagram below shows how each branch type relates to the others and which environment it targets. Time flows left to right.

Branch	T1	T2	T3	T4	T5	T6	T7	T8
main	o------	------	--------	--------	**<-- PR**	--[rel]--	**<-hotfix**	--o
qa	o------	**<-feat**	--tests--	**<-bugfix**	--tests--	**--> PR**	**<-hotfix**	--o
feature/*	branch	**-->o**	(merged)					
bugfix/*			branch	**-->o**	(merged)			
hotfix/*						branch	**-->main+qa**	(merged)
Env	**PROD**	**QA**	**QA**	**QA**	**QA**	**PROD**	**PROD+QA**	**PROD**

Legend: *o = branch point / merge commit | --> = direction of merge | <-- PR = pull request raised | [rel] = release tag*

© 2025 DevOps Shack. All rights reserved.

4. Deep-Dive: Each Branch Type

4.1 main — The Production Branch

Source of truth for production. Every commit on main represents code that is live (or ready to be live) in the PROD environment. This branch is protected — no direct pushes are ever permitted.

Rules & Conventions:

- Direct commits are BLOCKED — all changes arrive via Pull Request from qa or hotfix/*
- Requires minimum 2 approvals + all CI/CD checks green before merge
- Every merge to main automatically triggers a production deployment pipeline
- Each production release is tagged with a semantic version: v1.4.2
- Branch protection rules enforced via GitHub/GitLab branch policies
- Naming convention: main (always lowercase, never 'master')

4.2 qa — The Testing Integration Branch

The gatekeeper before production. The qa branch acts as the permanent integration branch for all development work. It is always deployed to the QA/Staging environment, where automated and manual testing runs continuously.

Rules & Conventions:

- Branched from: main (created once, lives permanently alongside main)
- All feature/* and bugfix/* branches merge INTO qa via Pull Request
- CI pipeline runs full integration + regression test suite on every merge
- QA environment is auto-deployed from qa on every merge (CD)
- QA testers perform manual exploratory testing against this deployment
- When qa is stable and approved, it is merged to main via a release PR
- Must be kept in sync with main after every production release

4.3 feature/* — New Development Branches

Short-lived, engineer-owned. Every new piece of functionality lives in a feature branch. Engineers work here in isolation, committing freely without impacting teammates.

Lifecycle:

- Created from: qa (git checkout -b feature/my-feature qa)
- Developer writes code, runs local unit tests, pushes to remote
- Opens a Pull Request targeting qa when work is complete

© 2025 DevOps Shack. All rights reserved.

- PR triggers: lint checks, unit tests, security scan (SAST), build validation
- At least 1 peer code review required before merge
- Merged INTO: qa via squash-merge (keeps qa history clean)
- Branch deleted immediately after successful merge
- Naming convention: feature/<ticket-id>-short-description e.g. feature/DS-421-user-auth

4.4 bugfix/* — QA Defect Fix Branches

Fix it where it was found. When the QA team raises a defect against code deployed to the QA environment, engineers create a bugfix branch directly from qa. The fix is tested in QA before it can ever reach production.

Lifecycle:

- Triggered by: defect ticket raised by QA team after testing on qa branch
- Created from: qa (git checkout -b bugfix/my-fix qa)
- Engineer diagnoses and patches the defect locally
- Pull Request raised against qa — peer-reviewed and CI-checked
- QA retests the specific defect on the QA environment post-merge
- Merged INTO: qa (never directly into main)
- Branch deleted after successful merge and defect closure
- Naming convention: bugfix/<ticket-id>-short-description e.g. bugfix/DS-512-login-null-pointer

Key distinction — bugfix vs hotfix: A bugfix is for a defect caught in the QA environment before it reaches production. A hotfix is for a defect already live in production that requires an emergency fix.

4.5 hotfix/* — Emergency Production Fix Branches

Production is broken — act fast. A hotfix branch bypasses the normal qa flow because the problem is already live and impacting real users. It is the only branch type that branches directly from main and merges directly back to main.

Lifecycle:

- Triggered by: P0/P1 incident in production — declared by on-call engineer or SRE
- Created from: main (git checkout -b hotfix/my-fix main)
- Fix is developed, reviewed by at least 1 senior engineer (expedited review)
- Pull Request raised against BOTH main and qa simultaneously
- CI/CD runs smoke tests; SRE approves deployment to production
- Merged INTO: main (immediately deployed to PROD) + qa (to keep branches in sync)
- Production release tagged with a patch version bump: v1.4.1 → v1.4.2

© 2025 DevOps Shack. All rights reserved.

- Branch deleted post-merge; incident post-mortem raised within 24h
- Naming convention: hotfix/<ticket-id>-short-description e.g. hotfix/INC-023-payment-timeout

© 2025 DevOps Shack. All rights reserved.

5. Merge Rules & Pull Request Policy

All merges are performed via Pull Requests (PRs) — never via direct push. The table below defines who can approve what and what automated gates must pass.

PR Type	From	Into	Approvals Needed	Automated Gates	Merge Strategy
Feature PR	feature/*	qa	1 peer reviewer	Lint, unit tests, SAST scan, build	Squash merge
Bugfix PR	bugfix/*	qa	1 peer reviewer	Lint, unit tests, regression suite	Squash merge
Release PR	qa	main	2 seniors + QA lead sign-off	Full regression, perf baseline, security scan	Merge commit (no squash)
Hotfix PR	hotfix/*	main + qa	1 senior + SRE	Smoke tests, critical path only (expedited)	Merge commit

6. Branch-to-Environment Mapping

The two environments used in this strategy map directly to branch deployment targets:

Environment	Branch(es)	Who Uses It
QA / Staging Cluster *(Dev + QA + PPD namespaces)*	• qa (permanent) • feature/* (pre-merge, local dev) • bugfix/* (pre-merge, QA retest)	• Developers (feature dev in dev ns) • QA Engineers (testing in qa/ppd ns) • Product Owners (UAT in ppd ns)
Production Cluster *(Live user traffic)*	• main (permanent) • hotfix/* (emergency, immediately post-merge)	• End users (live application) • SRE / Platform team (monitoring) • On-call engineers (incident response)

© 2025 DevOps Shack. All rights reserved.

© 2025 DevOps Shack. All rights reserved.

7. Step-by-Step Developer Workflows

7.1 Starting a New Feature

```
git checkout qa
git pull origin qa
git checkout -b feature/DS-421-user-auth
# ... develop and commit your work ...
git push origin feature/DS-421-user-auth
# Open Pull Request: feature/DS-421-user-auth  -->  qa
```

7.2 Fixing a QA Defect (Bugfix)

```
git checkout qa
git pull origin qa
git checkout -b bugfix/DS-512-login-null-pointer
# ... fix the defect and add regression test ...
git push origin bugfix/DS-512-login-null-pointer
# Open Pull Request: bugfix/DS-512-login-null-pointer  -->  qa
```

7.3 Releasing to Production (qa → main)

```
# QA team signs off, QA lead approves release
# Open Pull Request: qa  -->  main
# Requires 2 senior approvals + full CI gate green
# After merge — tag the release:
git checkout main && git pull
git tag -a v1.5.0 -m 'Release v1.5.0'
git push origin v1.5.0
```

7.4 Emergency Hotfix

```
# Incident declared — P0 in production
git checkout main
git pull origin main
git checkout -b hotfix/INC-023-payment-timeout
# ... apply fix ...
git push origin hotfix/INC-023-payment-timeout
# PR 1: hotfix/INC-023-payment-timeout  -->  main  (deploy to PROD)
# PR 2: hotfix/INC-023-payment-timeout  -->  qa   (keep in sync)
# Tag patch release: v1.5.1
```

© 2025 DevOps Shack. All rights reserved.

© 2025 DevOps Shack. All rights reserved.

8. Branch Naming Conventions

Consistent naming allows CI/CD pipelines to auto-detect branch type, apply the right gates, and deploy to the right namespace. All branch names must be lowercase with hyphens (no spaces, no camelCase).

Type	Pattern	Example
Feature	feature/<ticket-id>-<short-slug>	feature/DS-421-user-auth-oauth
Bugfix	bugfix/<ticket-id>-<short-slug>	bugfix/DS-512-login-null-pointer
Hotfix	hotfix/<ticket-id>-<short-slug>	hotfix/INC-023-payment-timeout
Release tag	vMAJOR.MINOR.PATCH	v2.1.4

9. Do's and Don'ts

Do (Best Practices)	Don't (Anti-patterns)
• Keep feature branches short-lived (< 3 days ideally) • Always pull latest qa before creating a new branch • Write a clear PR description linking the Jira/ticket • Delete branches immediately after merge • Tag every production release with a semantic version • Sync qa with main after every hotfix or release • Add a regression test for every bugfix	• Never push directly to main or qa • Never create a feature branch from main • Never create a bugfix branch from main (use qa) • Do not let branches live for weeks — they drift and cause conflicts • Do not merge a PR with failing CI gates — ever • Do not skip the hotfix → qa sync after an emergency fix • Do not combine multiple features into one branch

10. Summary

DevOps Shack Branching Strategy at a Glance

main is sacred — always production-ready, protected, tagged on every release.

qa is the integration hub — all development flows through here before touching production.

© 2025 DevOps Shack. All rights reserved.

feature/* branches from qa, merges back to qa — short-lived, one ticket per branch.

bugfix/* branches from qa, merges back to qa — for defects caught in testing, never in prod.

hotfix/* branches from main, merges to main AND qa — emergency only, always back-synced.

This model keeps production stable, testing isolated, and every code change traceable from ticket to tag.

DevOps Shack

Ship fast. Ship safe. Ship smart.

© 2025 DevOps Shack. All rights reserved.